

Praktikumsbericht

Canny Edge Detection auf der GPU

Matthias Stefan

Thomas Jurczyk

21. Juli 2026

Inhaltsverzeichnis

1	Voraussetzungen und Projektaufbau	3
1.1	Hardware-Spezifikationen	3
1.2	Projektstruktur	3
1.3	Build-System und Kompilierung	3
1.4	Messmethodologie	4
1.5	Verwandte Arbeiten	4
2	Algorithmus	6
2.1	Überblick	6
2.2	Schritte	6
2.2.1	Schritt 1: Gaussian Blur	6
2.2.2	Schritt 2: Sobel	6
2.2.3	Schritt 3: Non-Maximum Suppression	6
2.2.4	Schritt 4: Hysteresis	7
3	Implementierung	8
3.1	Schritt 1: Gaussian Blur	8
3.1.1	Naive Implementierung	8
3.1.2	Shared Memory Optimierung	9
3.1.3	Pinned Memory Optimierung	9
3.1.4	Warp-Level Optimierung	9
3.2	Schritt 2: Sobel	9
3.3	Schritt 3: Non-Maximum Suppression	9
3.3.1	Naive Implementierung	9
3.3.2	Shared Memory Optimierung	10
3.3.3	Bucket-Optimierung	10
3.4	Schritt 4: Hysteresis	11
4	Ergebnisse und Analyse	12
4.1	Gaussian Blur	12
4.1.1	Korrektheit	12
4.1.2	Performance	12
4.2	Schritt 2: Sobel	13

4.3	Schritt 3: Non-Maximum Suppression	13
4.3.1	Korrektheit	13
4.3.2	Performance	13
4.4	Schritt 4: Hysteresis	14
5	Fazit	15
5.1	Fazit	15
	Anhang	17
A	Gaussian Blur Messungen	17
B	Sobel Messungen	18
C	NMS Messungen	20

1 Voraussetzungen und Projektaufbau

1.1 Hardware-Spezifikationen

Die Implementierung wurde auf zwei unterschiedlichen Systemen entwickelt und gemessen. Die Hardware-Ausstattung unterscheidet sich dabei deutlich, was den späteren Vergleich der Ergebnisse interessant macht.

System: Matthias Stefan

Parameter	Wert
GPU	NVIDIA GeForce RTX 2060 (Laptop)
Compute Capability	7.5
Gesamter GPU-Speicher	5737 MB
L2-Cache	3072 KB
Streaming Multiprocessors (SM)	30
Max. Threads pro SM	1024
Warpgröße	32
Memory-Takt	5501 MHz
Memory-Busbreite	192 bit
Peak-Speicherbandbreite	264 GB/s
GPU-Takt	1350 MHz

Tabelle 1: GPU-Spezifikationen der RTX 2060 (Matthias Stefan)

1.2 Projektstruktur

Der Quellcode ist modular nach Pipeline-Stufen organisiert, wobei jede Stufe aus einem `.cu/.cuh`-Paar besteht, das die jeweiligen `execute()`- und `cleanup()`-Funktionen sowie die zugehörigen CUDA-Kernel kapselt. Zum Einlesen und Schreiben der Bilddaten wird die Header-only-Bibliothek `stb_image` eingesetzt.

```
/
├── run_debug.sh
├── run_release.sh
├── src/
│   ├── common.cuh
│   ├── gaussian.cu(h)
│   ├── main.cu
│   ├── nms.cu(h)
│   ├── sobel.cu(h)
│   └── stb_image(_write).h
```

1.3 Build-System und Kompilierung

Die Optimierungsstufe je Pipeline-Stufe wird zur Kompilierzeit über Preprocessor-Makros gewählt, wodurch für jede Konfiguration ein eigenes, spezialisiertes Binary ohne Laufzeit-Overhead entsteht. Die Stufen bauen dabei aufeinander auf, statt unabhängige Alternativen darzustellen: Bei Gaussian ersetzt Shared Memory zunächst die naive Faltung durch Tiling, während Pinned Memory zusätzlich den D→H-Transfer beschleunigt, sodass die höchste Stufe stets beides kombiniert.

`build_release.sh` übersetzt jede Übersetzungseinheit einzeln mit `nvcc` und reicht die Stufe über die Umgebungsvariablen `GAUSSIAN_OPT`, `SOBEL_OPT` und `NMS_OPT` als `-D-Flags` weiter, zum Beispiel:

```
GAUSSIAN_OPT=1 SOBEL_OPT=0 NMS_OPT=1 ./build_release.sh
```

Ohne Angabe greifen die Standardwerte (`GAUSSIAN_OPT=2`, `SOBEL_OPT=1`, `NMS_OPT=1`), jeweils die am weitesten optimierte Variante, während `WARMUP` (Standard: aktiviert) einen Aufwärm-Kerneldurchlauf vor der Zeitmessung steuert. Da separable Device-Compilation (`-rdc=true`) verwendet wird, ist zudem ein Device-Link-Schritt (`nvcc -dlink`) vor dem finalen Host-Link nötig, bei dem die Ziel-Architektur (`-arch=sm_75`) erneut angegeben werden muss, da `nvcc` sonst implizit auf eine ältere Architektur zurückfällt.

Analog erzeugt `build_debug.sh` eine Debug-Variante mit den Flags `-G -O0`, ohne `-use_fast_math`; alle berichteten Messungen stammen ausschließlich aus dem Release-Build. Das Ausführen der jeweils gebauten Binary übernehmen die separaten Skripte `run_release.sh` bzw. `run_debug.sh` mit fest hinterlegtem Ein- und Ausgabepfad.

1.4 Messmethodologie

Die Performance-Messungen wurden unter nicht-isolierten Bedingungen durchgeführt: Hintergrundprozesse, thermisches Throttling, dynamisches Power Management der Laptop-GPU, nicht garantiert geleerte Caches sowie OS-Scheduling-Interrupts bei $H \rightarrow D/D \rightarrow H$ -Transfers können die Ergebnisse beeinflusst haben. Die Ergebnisse sind daher als Richtwerte zu verstehen. Um Schwankungen zu minimieren, wurde jede Messung als Durchschnitt über 200 Runs ermittelt, wobei der erste Aufruf (Warm-up Run) separat ausgewiesen wird, da CUDA-Kontext und Page-Cache zu diesem Zeitpunkt noch nicht aufgewärmt sind. NCU-Profilung wurde mit 10 Kernel-Instanzen pro Report durchgeführt, der Warm-up Run wurde dabei ausgeschlossen.

1.5 Verwandte Arbeiten

Dieser Abschnitt stellt vier Arbeiten vor, die den Canny-Algorithmus auf GPU- bzw. GPU-nahen Architekturen parallelisieren, mit besonderem Fokus darauf, welche Speicheroptimierungen die jeweiligen Autoren für die Non-Maximum Suppression einsetzen und begründen.

Der in diesem Praktikum implementierte Algorithmus basiert auf dem von Canny [1] in *A Computational Approach to Edge Detection* vorgeschlagenen mehrstufigen Verfahren zur Kantendetektion, bestehend aus Gaussian Blur, Gradientenberechnung, Non-Maximum Suppression und Hysteresis Thresholding, dem die eigene Pipeline unmittelbar folgt.

Song et al. [4] implementieren in *A Parallel Canny Edge Detection Algorithm Based on OpenCL Acceleration* eine OpenCL-basierte Variante und begründen den Einsatz von Local Memory ausschließlich für die Faltungsstufen, da hier pro Pixel n^2 Nachbarn gelesen werden; für die Non-Maximum Suppression, die nur zwei Nachbarn pro Pixel benötigt, fehlt im Pseudocode dagegen jeder Lade- oder Kachelungsschritt.

Ogawa et al. [3] stellen in *Efficient Canny Edge Detection using a GPU* eine frühe CUDA-Implementierung vor, bei der die Non-Maximum Suppression ohne Kachelung direkt auf dem Global Memory arbeitet. Der eigentliche Schwerpunkt liegt auf einer stack-basierten Traversierung der nachgelagerten Hysteresis-Stufe, die im Gegensatz zu Ansätzen mit fester Traversierungstiefe auch lange, über mehrere Bildkacheln verlaufende Kantenketten vollständig erfasst.

Horvath et al. [2] implementieren in *Canny Edge Detection on GPU using CUDA* die Canny-Pipeline mit einem eigenen Kernel pro Stufe. Die Shared-Memory-Implementierung

der Non-Maximum-Suppression orientiert sich am Tiling-Ansatz des Sobel-Kernels, benötigt aufgrund der geringeren Nachbarschaftsgröße jedoch nur einen minimalen Halo-Rand. Mittels NSIGHT Compute identifizieren die Autoren zudem `hypot/atan2` als Hauptbottleneck ihres Sobel-Kernels; deren Ersatz durch konstante Zuweisungen reduzierte die Kernel-Laufzeit um den Faktor 8.0.

Yadav und Gupta [5] vergleichen in *Optimizing CUDA Kernels for High-Performance Canny Edge Detection* vier Optimierungsstufen anhand einer Roofline-Analyse, die sich jedoch ausschließlich auf den Gaussian-Filter-Kernel bezieht. Bemerkenswert ist, dass die Autoren die Non-Maximum Suppression konzeptionell nicht dem Shared-Memory-Ansatz, sondern der Warp-Level-Optimierung zuordnen, ohne diesen Kernel jedoch gesondert zu profilieren.

2 Algorithmus

2.1 Überblick

Der Canny-Algorithmus ist ein mehrstufiges Verfahren zur Kantendetektion in Graustufenbildern. Canny definiert drei Kriterien für einen optimalen Kantendetektor: gute Detektion, präzise Lokalisierung sowie exakt eine Antwort pro Kante, die auf eine Breite von einem Pixel supprimiert werden soll [1, S. 680]. Die Verarbeitung erfolgt in vier aufeinanderfolgenden Schritten, wobei der Output jedes Schritts als Input des nächsten dient (siehe Abbildung 1). Die einzelnen Schritte werden in den kommenden Abschnitten näher erläutert.

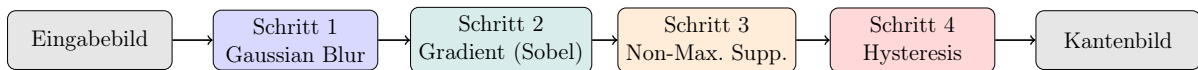


Abbildung 1: Canny Edge Detection Pipeline

2.2 Schritte

2.2.1 Schritt 1: Gaussian Blur

Um einen mathematisch optimalen Kantendetektor herzuleiten, trifft Canny eine Annahme über das Rauschen im Bild: „*We will assume that the image consists of the edge and additive white Gaussian noise*“ [1, S. 680]. Da dieses Rauschen fälschlicherweise als Kante detektiert werden kann, muss das Bild zunächst geglättet werden. Canny zeigt, dass der optimale Filter für dieses Rauschmodell durch die erste Ableitung einer Gaußfunktion approximiert werden kann [1, S. 687]:

$$G(x) = \exp\left(-\frac{x^2}{2\sigma^2}\right) \quad (1)$$

Der Parameter σ kontrolliert die Stärke der Glättung. Zwischen Rauschunterdrückung und Lokalisierungsgenauigkeit besteht jedoch ein grundlegender Zielkonflikt, den Canny als „*uncertainty principle*“ zwischen Detektion und Lokalisierung bezeichnet [1, S. 684].

2.2.2 Schritt 2: Sobel

2.2.3 Schritt 3: Non-Maximum Suppression

Schritt 2 liefert für jeden Pixel einen Gradientenbetrag, der die lokale Kantenstärke beschreibt, wobei die resultierenden Kanten an vielen Stellen breiter als ein Pixel sind. Non-Maximum Suppression soll diese breite Gradientenantwort im Idealfall auf eine Breite von einem Pixel reduzieren, indem entlang der Gradientenrichtung nur das lokale Maximum erhalten bleibt. Canny begründet diese Forderung mit dem Kriterium präziser Lokalisierung: „*The points marked as edge points by the operator should be as close as possible to the center of the true edge*“ [1, S. 680]. Formal entspricht der gesuchte Kantenpunkt einer Nullstelle der zweiten Richtungsableitung des geglätteten Bildes entlang der Gradientenrichtung n [1, S. 691], die aus der geglätteten Gradientenrichtung geschätzt wird, da sie nicht vorab bekannt ist.

In der diskreten Implementierung wird dies approximiert, indem die Gradientenmagnitude jedes Pixels mit den beiden direkten Nachbarn entlang der auf vier Klassen ($0^\circ/45^\circ/90^\circ/135^\circ$) quantisierten Gradientenrichtung verglichen wird. Ein Pixel überlebt die Suppression genau dann, wenn seine Magnitude größer oder gleich beiden Nachbarn ist, andernfalls wird er auf null gesetzt. Die Ein-Pixel-Breite wird dabei nur bei einem strikten lokalen Maximum tatsächlich erreicht, denn bei einem Plateau, also mehreren benachbarten Pixeln mit identischer Magnitude, erfüllen aufgrund des $\geq$ -Vergleichs alle beteiligten Pixel gleichzeitig das Überlebenskriterium, wodurch die Kante an dieser Stelle breiter als ein Pixel bleibt.

2.2.4 Schritt 4: Hysteresis

3 Implementierung

3.1 Schritt 1: Gaussian Blur

3.1.1 Naive Implementierung

Die Gaußgewichte werden auf der CPU vorberechnet und normalisiert (Summe 1, Gleichung 1), einmalig per `cudaMemcpy` auf die GPU übertragen und stehen dort als Read-only-Buffer im globalen Speicher zur Verfügung. Abbildung 2 zeigt das resultierende 3×3 Gewichtsraaster für $\sigma = 1.0$.

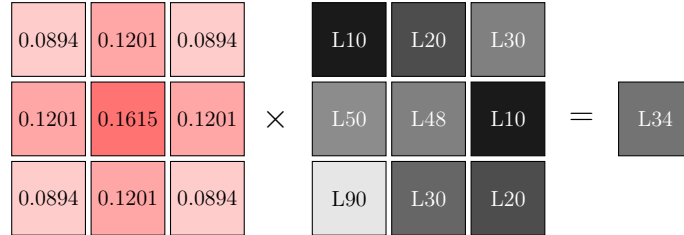


Abbildung 2: Gaussian Blur, gewichteter Mittelwert am Beispielpixel $L48 \rightarrow L34$

Listing 1 zeigt den naiven Kernel `gaussian_filter()`: jeder Thread berechnet einen Ausgabepixel als gewichtete Summe seiner $(2r + 1)^2$ Nachbarn, mit Zero-Padding am Bildrand. Die Threads eines Blocks greifen dabei in y-Richtung auf Adressen mit Abstand `width` zu, wodurch die Zugriffe auf den globalen Speicher nicht coalesced erfolgen, was die in Abschnitt 3.1.2 beschriebene Shared Memory Optimierung motiviert; die quantitative Auswertung findet sich in Abschnitt 4.1.

```

1  __global__ void gaussian_filter(const uint8_t* src_buffer, const int32_t width,
2  const int32_t height, const float* weights, const uint32_t blur_radius,
3  uint8_t* out_dst_buffer)
4  {
5      // globale Pixelposition, out-of-bounds Threads verwerfen
6      const auto x = static_cast<int32_t>(blockIdx.x * blockDim.x + threadIdx.x);
7      const auto y = static_cast<int32_t>(blockIdx.y * blockDim.y + threadIdx.y);
8      if (x >= width || y >= height)
9          return;
10
11     // Gewichtete Summe der Nachbarpixel akkumulieren (Zero-Padding am Rand)
12     float color = 0.0f;
13     const auto radius = static_cast<int32_t>(blur_radius);
14     for (int32_t y_ = -radius; y_ <= radius; y_++)
15     {
16         for (int32_t x_ = -radius; x_ <= radius; x_++)
17         {
18             // Pixel ausserhalb der Bildgrenzen mit 0 auffuellen
19             uint8_t pixel = 0;
20             if (x + x_ >= 0 && x + x_ < width && y + y_ >= 0 && y + y_ < height)
21                 pixel = src_buffer[(y + y_) * width + (x + x_)];
22
23             // strided access mit Abstand width
24             const uint32_t weight_idx =
25                 (y_ + radius) * (blur_radius * 2 + 1) + (x_ + radius);
26             color += pixel * weights[weight_idx];
27         }
28     }
29
30     out_dst_buffer[y * width + x] = static_cast<uint8_t>(color);
31 }

```

Listing 1: `gaussian_filter` Kernel

3.1.2 Shared Memory Optimierung

Bei einem Kernel der Größe $(2r + 1)^2$ liest jeder Thread bis zu $(2r + 1)^2$ Pixel redundant aus dem DRAM. [2, S. 2] adressieren dieses Problem durch Shared Memory Tiling, bei dem Threads eines Blocks kooperativ eine Bildkachel inklusive Halo-Rand der Größe $(\text{BLOCK_SIZE} + 2 \cdot \text{BLUR_RADIUS})^2$ in den On-Chip Speicher laden, bevor `__syncthreads()` die vollständige Ladephase sicherstellt. Die Gaußgewichte werden zusätzlich in `__constant__` Memory ausgelagert, da alle Threads dieselben Gewichte lesen und dieser Speicher für uniforme Zugriffe hardwareseitig gecacht wird. Die Auswirkung dieser Optimierung auf Laufzeit und Speicherdurchsatz wird in Abschnitt 4.1 ausgewertet.

3.1.3 Pinned Memory Optimierung

Pinned Memory (Page-locked Memory) ist Host-seitiger Speicher, der vom Betriebssystem nicht ausgelagert werden kann, wodurch direkte DMA-Transfers ohne zwischengeschalteten CPU-Cache möglich sind. Die Änderung gegenüber der Shared Memory Implementierung beschränkt sich auf die Host-seitige Allokation (`cudaMallocHost()` statt `malloc()`); der Kernel bleibt identisch, der erwartete Vorteil zeigt sich primär beim Device-to-Host-Transfer (Abschnitt 4.1).

3.1.4 Warp-Level Optimierung

Als vierte Stufe wurden Warp-Level Ininsics (`__shfl_sync()`) evaluiert, die Datenaustausch innerhalb eines Warps ohne Shared Memory ermöglichen. Für 2D-Stencil-Operationen wie den Gaussian Blur ist dieser Ansatz jedoch ungeeignet, da `__shfl_sync()` primär für 1D-Reduktionen konzipiert ist, was [5] bestätigen (79.29 % vs. 83.33 % Compute/Memory Throughput bei Shared Memory). Eine eigene Implementierung wurde daher nicht vorgenommen.

3.2 Schritt 2: Sobel

3.3 Schritt 3: Non-Maximum Suppression

3.3.1 Naive Implementierung

Die Non-Maximum Suppression wird im CUDA-Kernel `non_maximum_suppression()` umgesetzt, wobei jeder Thread für genau einen Pixel zuständig ist. Als Input erhält der Kernel den Gradientenbetrag (`magnitude_buffer`) sowie die Gradientenrichtung in Radiant (`direction_buffer`) aus Schritt 3.2.

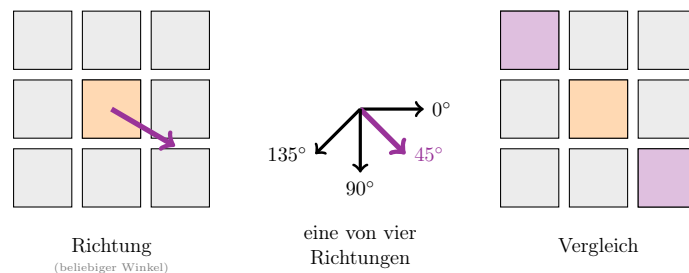


Abbildung 3: Non-Maximum Suppression: Die Gradientenrichtung wird auf eine von vier Achsen quantisiert (0°, 45°, 90°, 135°). Anschließend wird der Pixel mit den beiden farbig markierten Nachbarn entlang der gewählten Achse verglichen.

Die Gradientenrichtung wird zunächst von Radiant in Grad umgerechnet und auf den Bereich $[0^\circ, 180^\circ)$ gefaltet, da eine Kante keine Orientierung besitzt und z. B. 170° und -10° dieselbe Achse beschreiben. Anschließend wird die Richtung auf eine von vier Klassen quantisiert, die den vier möglichen Achsen im diskreten Pixelraster entsprechen (Abbildung 3). Je nach Richtungsklasse werden die Offsets (dx, dy) zum Nachbarpixel bestimmt; der Pixel überlebt die Suppression genau dann, wenn seine Magnitude größer oder gleich beiden Nachbarn entlang dieser Achse ist, andernfalls wird er auf null gesetzt (Listing 2).

```

1  __global__ void non_maximum_suppression(
2      const uint8_t* magnitude_buffer,
3      const float* direction_buffer,
4      const int32_t width,
5      const int32_t height,
6      uint8_t* out_nms_buffer)
7  {
8      // Thread-Position bestimmen, out-of-bounds Threads verwerfen
9      ...
10
11     const uint8_t magnitude = magnitude_buffer[y * width + x];
12     const float direction = direction_buffer[y * width + x];
13
14     // Richtung von Radiant in Grad umrechnen und auf [0, 180) falten
15     ...
16
17     // Richtung auf eine von vier Klassen quantisieren, Offset (dx, dy) bestimmen
18     int32_t dx, dy;
19     if (deg < 22.5f)      { dx = 1; dy = 0; }
20     else if (deg < 67.5f) { dx = 1; dy = -1; }
21     else if (deg < 112.5f) { dx = 0; dy = 1; }
22     else if (deg < 157.5f) { dx = 1; dy = 1; }
23     else                 { dx = 1; dy = 0; }
24
25     const uint8_t neighbor_a =
26         sample_clamped(magnitude_buffer, width, height, x + dx, y + dy);
27     const uint8_t neighbor_b =
28         sample_clamped(magnitude_buffer, width, height, x - dx, y - dy);
29
30     out_nms_buffer[y * width + x] =
31         (magnitude >= neighbor_a && magnitude >= neighbor_b) ? magnitude : 0;
32 }

```

Listing 2: non_maximum_suppression Kernel (gekürzt)

3.3.2 Shared Memory Optimierung

Die Shared-Memory-Variante lädt zunächst kooperativ eine Bildkachel inklusive eines Halo-Rands der Breite 1 in den On-Chip Speicher, statt für jeden Nachbarn einen eigenen Zugriff auf den globalen Speicher auszuführen. Der Halo-Rand fällt dabei deutlich schmaler aus als bei Gaussian Blur (Abschnitt 3.1.2), da Non-Maximum Suppression stets nur die unmittelbare 8er-Nachbarschaft benötigt. Da alle Threads eines Blocks am kollektiven Laden beteiligt sein müssen, erfolgt die Grenzprüfung erst nach `__syncthreads()`, um einen Deadlock durch nicht erreichte Threads zu vermeiden. Magnitude und Nachbarn werden anschließend direkt aus der Kachel statt aus dem globalen Speicher gelesen. Die quantitative Auswertung findet sich in Abschnitt 4.3.

3.3.3 Bucket-Optimierung

Als letzte Optimierungsmöglichkeit verwendet diese Stufe einen vom Sobel-Kernel bereits vorquantisierten Richtungs-Bucket $\in \{0, 1, 2, 3\}$ statt Radiant-Werten, wodurch Gradumrechnung und Klassenbestimmung entfallen; die (dx, dy) -Offsets kommen stattdessen aus

einer Lookup-Tabelle im Constant Memory. Die Magnitude bleibt gekachelt, da sie mehrfach gelesen wird; der Bucket-Wert dagegen wird direkt aus dem globalen Speicher gelesen, da hier keine Datenwiederverwendung zwischen Threads stattfindet.

3.4 Schritt 4: Hysteresis

4 Ergebnisse und Analyse

4.1 Gaussian Blur

4.1.1 Korrektheit

Abbildung 4 zeigt das Ergebnis der Filterung im Vergleich zum Originalbild.



Abbildung 4: Vergleich Original (links) vs. Gaussian Blur (rechts, $\sigma = 25$, `BLUR_RADIUS=12`). Die Parameter wurden bewusst extrem gewählt, um den Effekt deutlicher sichtbar zu machen, in der Pipeline wird ein deutlich sanfterer Filter verwendet.

4.1.2 Performance

Das NCU-Profil der naiven Implementierung zeigt, dass 47 % aller Global Memory Zugriffe uncoalesced sind, was die in Abschnitt 3.1.2 beschriebene Shared Memory Optimierung motiviert.

Der gemessene Speedup hängt stark vom `BLUR_RADIUS` ab (Abbildung 5). Bei $r = 2$ überwiegt der `__syncthreads()`-Overhead, sodass die Kernel-Laufzeit bei nahezu identischem SM Throughput nur marginal sinkt. Bei $r = 9$ kehrt sich das Verhältnis um, da die 361 Faltungsoperationen pro Pixel den redundanten DRAM-Traffic zum dominanten Faktor machen, wodurch Shared Memory einen Speedup von $3.0\times$ bei deutlich gesteigertem SM Throughput erzielt.

Die Pinned-Memory-Variante bestätigt, dass Compute Throughput und Occupancy bei $r = 9$ praktisch identisch zur Shared Memory Variante sind, da der Kernel selbst unverändert bleibt. Der Vorteil zeigt sich ausschließlich im D→H-Transfer, der bei $r = 9$ von 2.04 ms auf 1.37 ms sinkt. Die vollständigen Messdaten sind in Anhang A (S. 17) aufgeführt.

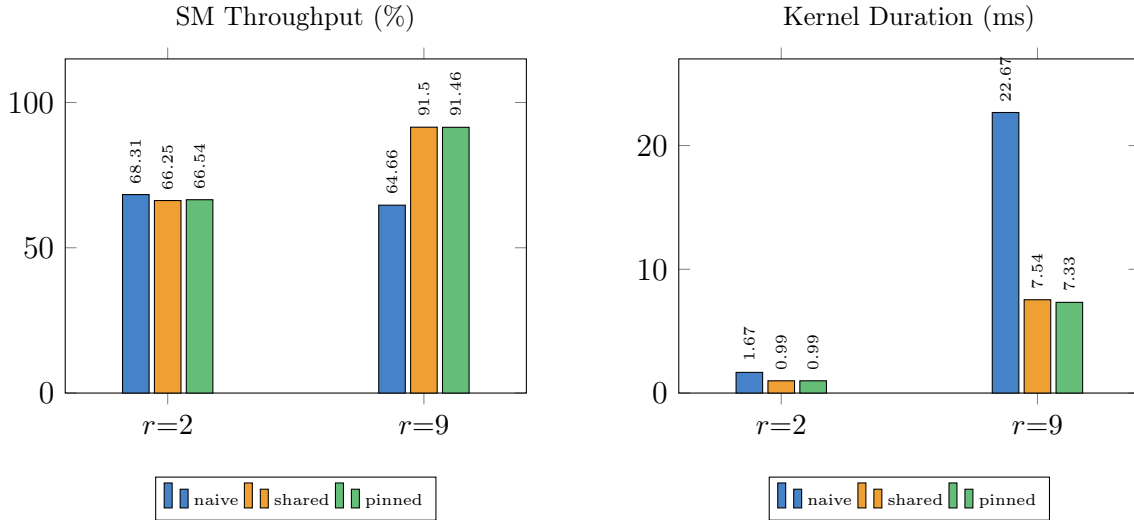


Abbildung 5: NCU-Profilung, Vergleich naive/shared/pinned bei $r=2$ und $r=9$ (200 Runs Avg).

4.2 Schritt 2: Sobel

4.3 Schritt 3: Non-Maximum Suppression

4.3.1 Korrektheit

Abbildung 6 zeigt einen Detailausschnitt des Ergebnisses der Non-Maximum Suppression im Vergleich zur Gradientenmagnitude aus Schritt 3.2: Die zuvor mehrere Pixel breite Gradientenantwort wird auf eine Breite von einem Pixel reduziert, mit Ausnahme lokaler Plateaus (vgl. Abschnitt 3.3).



Abbildung 6: Detailausschnitt: Vergleich Gradientenmagnitude (links) vs. Non-Maximum Suppression (rechts).

4.3.2 Performance

Das NCU-Profilung zeigt, dass der NMS-Kernel in allen drei Varianten sowohl beim Compute- als auch beim Memory-Throughput im Bereich von rund 31 % respektive 25 %

verbleibt (Tabelle 9), der Kernel ist also weder rechen- noch bandbreitengebunden, sondern latenzgebunden. Jeder Thread liest lediglich seinen eigenen Magnitudenwert sowie die zwei Nachbarn entlang der Gradientenrichtung, weshalb der redundante DRAM-Traffic, der beim Gaussian Blur die Shared Memory Optimierung motiviert, hier weitgehend entfällt.

Entsprechend zeigt Abbildung 7 ein zum Gaussian Blur gegenläufiges Bild. Der SM Throughput bleibt über naive, shared und pinned praktisch unverändert, die Shared-Memory-Variante hebt die Auslastung also nicht an. Gleichzeitig steigt die Kernel-Laufzeit von 0.576 ms auf 0.879 ms, da das kooperative Laden der Kachel und die `__syncthreads()`-Barriere Overhead erzeugen, dem bei einem so kleinen Stencil keine nennenswerte Einsparung gegenübersteht.

Das Profiling bestätigt diese Deutung im Detail: Die Shared-Memory-Variante senkt zwar den DRAM Throughput von 26.15 % auf 17.43 % und hebt die Occupancy von 85.10 % auf 93.37 %, doch da der Kernel nicht bandbreitengebunden ist, übersetzt sich diese Einsparung nicht in eine kürzere Laufzeit. Die naive Variante profitiert zudem von einer L1-Hit-Rate von 50.44 %, die in der Shared-Memory-Variante auf 23.32 % fällt, da die Zwischenpufferung in Shared Memory den L1-Pfad umgeht.

Der vorquantisierte Bucket-Richtungspuffer (Tabelle 8) senkt die naive Kernel-Laufzeit gegenüber dem Float-Puffer von 0.633 ms auf 0.576 ms, da pro Thread die Umrechnung von Radiant in Grad entfällt und statt eines `float` nur ein `uint8_t` gelesen wird.

Bemerkenswert ist, dass keine der herangezogenen Referenzarbeiten einen quantifizierten Shared-Memory-gegen-naiv-Vergleich speziell für die Non-Maximum Suppression liefert. Für diese Pipeline-Stufe zahlt sich Shared Memory Tiling, anders als beim Gaussian Blur, nicht aus.

Die Pinned-Memory-Variante lässt den Kernel unverändert und wirkt sich, wie beim Gaussian Blur, primär auf den Device-to-Host-Transfer aus, der von 1.925 ms auf 1.640 ms sinkt. Die vollständigen Messdaten sind in Anhang C (S. 20) aufgeführt.

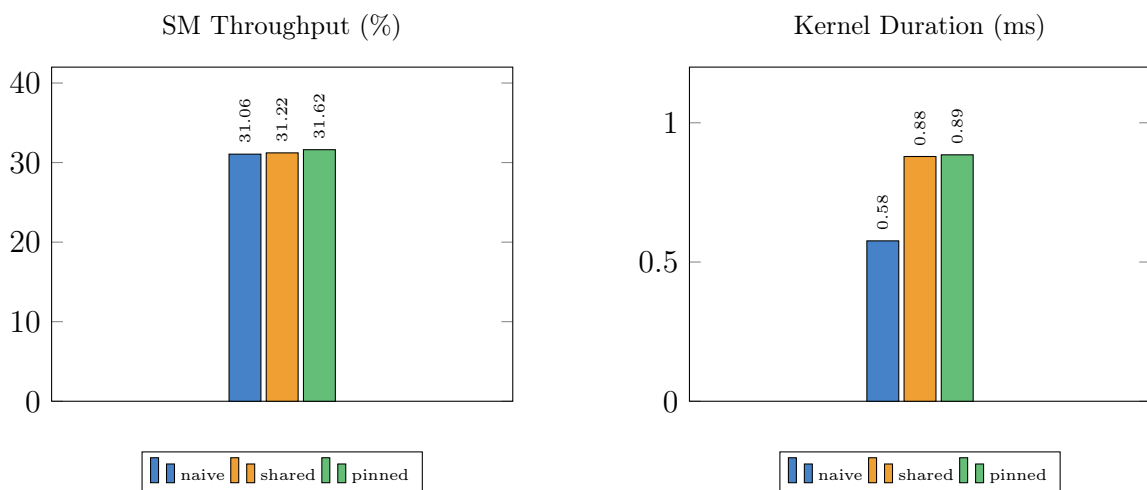


Abbildung 7: NMS-Profilung, SOBEL_OPT=4 (Bucket-Richtung). SM Throughput aus NCU (10 Runs), Kernel Duration aus cudaEvent-Messung (200 Runs Avg).

4.4 Schritt 4: Hysteresis

5 Fazit

5.1 Fazit

Literatur

- [1] John Canny. A computational approach to edge detection. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, PAMI-8(6):679–698, November 1986.
- [2] Matthew Horvath, Michael Bowers, and Shadi Alawneh. Canny edge detection on gpu using cuda. In *2023 IEEE 13th Annual Computing and Communication Workshop and Conference (CCWC)*, pages 0419–0425, 2023.
- [3] Kohei Ogawa, Yasuaki Ito, and Koji Nakano. Efficient canny edge detection using a GPU. In *2010 First International Conference on Networking and Computing*, pages 279–280, 2010.
- [4] Yupu Song, Cailin Li, Shiyang Xiao, Qinglei Zhou, and Han Xiao. A parallel canny edge detection algorithm based on OpenCL acceleration. *PLoS ONE*, 19(1):e0292345, January 2024.
- [5] Rishita Yadav and Harshul Gupta. Optimizing CUDA kernels for high-performance canny edge detection. *International Journal of Science, Engineering and Technology*, 13(4), 2025.

Anhang

A Gaussian Blur Messungen

Metrik	naive		shared		pinned	
	MS	TJ	MS	TJ	MS	TJ
<i>Warm-up Run</i>						
H→D [ms]	3.062	0.000	5.660	0.000	2.709	0.000
Kernel [ms]	2.606	0.000	1.697	0.000	1.688	0.000
D→H [ms]	7.631	0.000	7.020	0.000	2.502	0.000
Total [ms]	13.299	0.000	14.377	0.000	6.899	0.000
<i>200 Runs Avg</i>						
H→D [ms]	3.548	0.000	3.475	0.000	3.135	0.000
Kernel [ms]	1.669	0.000	0.990	0.000	0.987	0.000
D→H [ms]	3.068	0.000	3.001	0.000	1.740	0.000
Total [ms]	8.286	0.000	7.466	0.000	5.862	0.000

Tabelle 2: cudaEvent-Messungen, BLUR_RADIUS = 2. MS: Matthias/Stefan, TJ: Thomas Jurczyk.

Metrik (NCU, 10 Runs Avg)	naive	shared	pinned
Compute (SM) Throughput [%]	68.31	66.25	66.54
Memory Throughput [%]	61.10	66.25	66.54
DRAM Throughput [%]	5.81	8.42	7.60
Achieved Occupancy [%]	76.50	92.67	92.65
Warp Cycles Per Issued Instruction	8.79	16.46	16.33
L1/TEX Hit Rate [%]	92.94	27.32	27.29
L2 Hit Rate [%]	87.75	88.17	89.12

Tabelle 3: NCU-Profilung, BLUR_RADIUS = 2

Metrik	naive		shared		pinned	
	MS	TJ	MS	TJ	MS	TJ
<i>Warm-up Run</i>						
H→D [ms]	2.589	0.000	1.757	0.000	1.781	0.000
Kernel [ms]	35.700	0.000	12.575	0.000	13.182	0.000
D→H [ms]	6.035	0.000	6.183	0.000	1.836	0.000
Total [ms]	44.324	0.000	20.515	0.000	16.799	0.000
<i>200 Runs Avg</i>						
H→D [ms]	2.512	0.000	2.375	0.000	2.057	0.000
Kernel [ms]	22.669	0.000	7.537	0.000	7.325	0.000
D→H [ms]	2.142	0.000	2.036	0.000	1.366	0.000
Total [ms]	27.308	0.000	11.948	0.000	10.747	0.000

Tabelle 4: cudaEvent-Messungen, BLUR_RADIUS = 9. MS: Matthias Stefan, TJ: Thomas Jurczyk.

Metrik (NCU, 10 Runs Avg)	naive	shared	pinned
Compute (SM) Throughput [%]	64.66	91.50	91.46
Memory Throughput [%]	64.66	91.50	91.46
DRAM Throughput [%]	1.06	1.69	1.79
Achieved Occupancy [%]	95.52	94.84	94.83
Warp Cycles Per Issued Instruction	12.56	13.78	13.77
L1/TEX Hit Rate [%]	99.27	45.55	45.54
L2 Hit Rate [%]	63.36	82.96	87.35

Tabelle 5: NCU-Profilung, BLUR_RADIUS = 9

B Sobel Messungen

Metrik	naive		shared		optimized		pinned		bucket	
	MS	TJ	MS	TJ	MS	TJ	MS	TJ	MS	TJ
<i>Warm-up Run</i>										
H→D [ms]	1.835	0.000	3.372	0.000	3.030	0.000	1.293	0.000	1.400	0.000
Kernel [ms]	0.717	0.000	1.062	0.000	0.636	0.000	0.539	0.000	0.466	0.000
D→H [ms]	32.075	0.000	30.810	0.000	7.353	0.000	7.794	0.000	11.478	0.000
Total [ms]	34.626	0.000	35.244	0.000	11.019	0.000	9.626	0.000	13.344	0.000
<i>200 Runs Avg</i>										
H→D [ms]	1.601	0.000	1.715	0.000	1.648	0.000	1.596	0.000	1.926	0.000
Kernel [ms]	0.567	0.000	1.000	0.000	0.566	0.000	0.569	0.000	0.453	0.000
D→H [ms]	27.348	0.000	27.210	0.000	7.651	0.000	7.592	0.000	3.720	0.000
Total [ms]	29.516	0.000	29.925	0.000	9.865	0.000	9.757	0.000	6.099	0.000

Tabelle 6: Sobel-Filter, cudaEvent-Messungen. MS: Matthias Stefan, TJ: Thomas Jurczyk.

C NMS Messungen

Metrik	naive		shared		pinned	
	MS	TJ	MS	TJ	MS	TJ
<i>Warm-up Run</i>						
H→D [ms]	10.035	0.000	14.527	0.000	6.178	0.000
Kernel [ms]	0.603	0.000	1.089	0.000	1.002	0.000
D→H [ms]	6.540	0.000	7.319	0.000	1.315	0.000
Total [ms]	17.178	0.000	22.935	0.000	8.496	0.000
<i>200 Runs Avg</i>						
H→D [ms]	9.865	0.000	13.759	0.000	6.379	0.000
Kernel [ms]	0.633	0.000	1.015	0.000	1.009	0.000
D→H [ms]	1.925	0.000	2.678	0.000	1.361	0.000
Total [ms]	12.423	0.000	17.453	0.000	8.749	0.000

Tabelle 7: NMS mit Float-Richtungspuffer (SOBEL_OPT=2).
cudaEvent-Messungen. MS: Matthias Stefan, TJ: Thomas Jurczyk.

Metrik	naive		shared		pinned	
	MS	TJ	MS	TJ	MS	TJ
<i>Warm-up Run</i>						
H→D [ms]	3.778	0.000	5.521	0.000	2.484	0.000
Kernel [ms]	0.571	0.000	1.037	0.000	0.881	0.000
D→H [ms]	6.416	0.000	9.325	0.000	1.455	0.000
Total [ms]	10.765	0.000	15.883	0.000	4.819	0.000
<i>200 Runs Avg</i>						
H→D [ms]	4.020	0.000	7.221	0.000	3.048	0.000
Kernel [ms]	0.576	0.000	0.879	0.000	0.885	0.000
D→H [ms]	1.925	0.000	3.525	0.000	1.640	0.000
Total [ms]	6.521	0.000	11.625	0.000	5.572	0.000

Tabelle 8: NMS mit vorquantisiertem Bucket-Richtungspuffer (SOBEL_OPT=4). cudaEvent-Messungen.

MS: Matthias Stefan, TJ: Thomas Jurczyk.

Metrik (NCU, 10 Runs Avg)	naive	shared	pinned
Compute (SM) Throughput [%]	31.06	31.22	31.62
Memory Throughput [%]	26.93	24.65	24.96
DRAM Throughput [%]	26.15	17.43	18.20
Achieved Occupancy [%]	85.10	93.37	90.21
Warp Cycles Per Issued Instruction	31.17	22.51	23.06
L1/TEX Hit Rate [%]	50.44	23.32	23.23
L2 Hit Rate [%]	85.18	85.64	85.81

Tabelle 9: NCU-Profilung der NMS-Kernel, SOBEL_OPT = 4 (Bucket-Richtung). 10 Runs, Warm-up ausgeschlossen.